COMPUTER VISION COURSE PROJECT SUGGESTIONS

Orientation: Fundamental Computer Vision – with an emphasis on Digital Image Processing
With an appendix containing a sample course-project report template

Document Objective	Provide suggested project topics, application directions, expected deliverables, recommended pipelines, datasets, and criteria for 20 projects.
How to Choose and Execute a Project	Students select one project as their official assignment; they may choose either a research-oriented direction (method comparison or method development) or an application-oriented direction (packaging the work into a system with a clear output).

General Principles for Project Implementation

- Each project group may have up to 03 members, and the project may not be carried out individually.
- Each project must address a clearly defined application problem, rather than stopping at testing algorithms on a benchmark.
- At least one main task must be compared using two or more classical methods; comparison between classical CV and deep/learning-based methods is encouraged where appropriate.
- The report must include error analysis and both quantitative and qualitative discussion, not merely the best numerical result.
- The final product should take the form of an executable script, a notebook with a clear workflow, or a minimal interface demonstrating application value.

1. Mapping between Course Knowledge and Project Tasks

Knowledge	Tasks Appearing in the Project	Example Projects
Image representation, color channels, color spaces	Image reading, RGB/HSV/Lab conversion, color normalization, histogram analysis	Document scanner, leaf disease, skin lesion, fruit grading
Image filtering and denoising	Gaussian/median/bilateral filtering, smoothing before segmentation	Cell counting, lane detection, road damage
Convolution and edge detection	Sobel, Canny, gradient maps, structural detection	Document scanner, lane detection, road damage, plate detection
Thresholding and morphology	Foreground extraction, hole filling, region connection, separation of touching regions	Coin counting, cell counting, skin lesion, license plate
Connected components and contour analysis	Object counting, area measurement, shape analysis, enclosing regions	Coin counting, fruit grading, waste sorting, skin lesion
Feature detection / matching	Keypoint detection, descriptor matching, homography	Panorama stitching
Clustering / segmentation	K-means, watershed, region refinement	Leaf disease, skin lesion, aerial building extraction
Feature engineering	Color histogram, texture, shape, LBP/GLCM/HOG	Fabric defect, surface defect, traffic sign, gesture recognition

Classification / recognition	SVM, kNN, random forest, lightweight CNN	Waste classification, traffic sign, parking occupancy, food recognition
System evaluation	Accuracy, IoU, Dice, precision-recall, MAE, runtime	All projects

2. General Requirements for All Projects

- Provide a description of the problem, target users, application context, and the reasons why the problem has practical significance.
- Build a clear pipeline in the following sequence: Input → Preprocessing → Segmentation/Detection → Feature/Representation → Decision → Evaluation → Visualization.
- Describe the data: source, number of samples, labeling conventions, train/validation/test split, and data-cleaning criteria.
- Compare at least two methods for one main stage; clearly state the baseline and the improvement, or the baseline and the alternative method.
- The analysis of results must include both quantitative and qualitative aspects, with tables, illustrative figures, and discussion of errors/limitations.
- Submission requirements:
  - 01 report prepared according to the template provided in the appendix
  - The complete source code must be provided on Kaggle, GitHub, or Drive, together with the dataset in uncompressed form, properly organized and accompanied by an instruction file describing the data and experimental procedure so that the project can be executed for verification. If the dataset is too large, the program must include code to automatically download and organize the data so that it can be executed.
  - 01 slide presentation
  - Note: Do not submit compressed files.

3. Detailed List of 20 Projects

The topics below are only suggestions. Students may choose any topic of their own, but they must use all the knowledge learned in the course, satisfy all the requirements above, and complete a workload equivalent to the projects listed below.

Project 1. Automatic Coin Counting and Coin-Type Estimation

Application direction. Retail cashier support / vending-machine intake / educational counting assistant.
Required outcome. Build a system that counts coins in a single image, localizes each coin, and optionally estimates denomination or size group under varying lighting and mild overlap.

System Requirement Specification	• Count all visible coins in each image and display the count on the output image. • Mark each detected coin using a circle, contour, or bounding shape. • Compare at least two detection/segmentation strategies on the same test set. • Handle at least one difficult condition: overlap, cluttered background, or nonuniform lighting. • Produce a simple usage interface: command-line batch mode or minimal GUI.

Suggested Pipeline	1. Image acquisition and color normalization 2. Noise reduction and contrast enhancement 3. Foreground segmentation 4. Instance separation for touching coins 5. Circle/region detection 6. Counting and optional denomination estimation 7. Result visualization and error logging
Required Data	Self-collected dataset, 200–500 images. Include different backgrounds, shadows, scales, and partial overlap. A small validation set must be manually annotated with coin count and approximate circle/region locations.
Recommended Method Comparisons	• Segmentation: global threshold vs adaptive threshold vs K-means • Instance separation: distance transform + watershed vs contour splitting • Detection: Hough Circle vs contour-based ellipse/circle fitting
Required Analysis and Evaluation in the Report	Report counting accuracy, missed detections, false detections, and runtime per image. Analyze failure cases such as touching coins, specular highlights, or shadows. Discuss why one method is more stable under lighting variation or overlap.

Project 2. Automatic Document Scanner and Perspective Rectification

Application direction. Mobile document digitization for assignments, forms, receipts, and OCR preprocessing.

Required outcome. Create a system that converts a handheld photograph of a paper document into a flattened, readable scan with corrected perspective and improved contrast.

System Requirement Specification	• Automatically find the document boundary and estimate the four corners. • Rectify perspective and generate a readable output image. • Apply at least two different binarization/enhancement approaches and compare them. • Support batch processing for a folder of images. • Demonstrate robustness on non-ideal capture conditions.
Suggested Pipeline	1. Image preprocessing 2. Edge extraction 3. Document contour detection 4. Corner ordering and quadrilateral validation 5. Perspective transform 6. Contrast enhancement and binarization 7. Output quality check and save
Required Data	Self-collected dataset of at least 150 images. Include different paper sizes, desk textures, lighting conditions, skew angles, and partial shadows. Optional public data such as Doc3D may be used for stress testing.
Recommended Method Comparisons	• Boundary detection: Canny+largest contour vs line-based quadrilateral detection • Binarization: Otsu vs adaptive threshold vs contrast-limited preprocessing • Enhancement: grayscale only vs color-preserving scan
Required Analysis and Evaluation in the Report	Evaluate boundary detection success rate, readability, and geometric correction quality. Include qualitative comparison panels and discuss errors when background edges are strong or paper color is non-white. If OCR is added, report OCR accuracy before and after scanning.

Project 3. Leaf Disease Analysis for Smart Agriculture

Application direction. Field support tool for early crop disease screening from smartphone images.
Required outcome. Develop a system that segments the leaf, identifies diseased regions, and predicts the disease category or healthy/diseased status.

System Requirement Specification	• Segment the leaf from the background or justify a full-image approach. • Highlight likely diseased regions or produce a saliency-style explanation. • Compare at least one classical pipeline against one learning-based method. • Return disease label/confidence or healthy/diseased decision. • Provide a user-oriented output suitable for non-expert use.
Suggested Pipeline	1. Leaf/background separation 2. Color correction and shadow handling 3. Disease-region segmentation 4. Feature extraction or learned representation 5. Classification 6. Visualization with highlighted affected area and predicted label
Required Data	PlantVillage or a similar public dataset, optionally combined with a small self-captured set under natural lighting. Use at least 1,000 labeled images if classification is performed.
Recommended Method Comparisons	• Segmentation: threshold in HSV/Lab vs K-means vs contour-based leaf extraction • Features/classifier: color histogram + texture + SVM vs random forest vs lightweight CNN • Optional: compare cropped-leaf classification vs full-image classification
Required Analysis and Evaluation in the Report	Report accuracy, macro-F1, and class-wise behavior. Analyze confusion among visually similar diseases and sensitivity to lighting or leaf orientation. Discuss whether segmentation improves classification compared with full-image baselines.

Project 4. Microscopy Cell Counting and Instance Separation

Application direction. Laboratory automation support for cell quantification in microscopic images.
Required outcome. Build a system that detects or segments cells, separates touching instances, and estimates the total count per image.

System Requirement Specification	• Count cells in each image and visualize instance boundaries or centers. • Include a specific strategy for touching or clustered cells. • Compare at least two instance-separation approaches. • Export per-image count and aggregate statistics. • Use a quantitative evaluation protocol on a held-out set.
Suggested Pipeline	1. Noise suppression and contrast normalization 2. Foreground segmentation 3. Touching-cell separation 4. Cell localization and counting 5. Optional cell-type grouping 6. Visualization and statistics export
Required Data	BCCD or another microscopy dataset, optionally augmented with a self-curated subset. Use enough images to evaluate counting and segmentation under different densities.
Recommended Method Comparisons	• Segmentation: threshold/Otsu vs clustering • Separation: morphology + distance transform + watershed vs contour

	heuristics • Counting: connected components vs detected centers
Required Analysis and Evaluation in the Report	Measure count error, precision/recall for detected cells, and IoU/Dice if masks are available. Pay special attention to clustered cells, stain variation, and debris. Explain where over-segmentation or under-segmentation occurs.

Project 5. Skin Lesion Segmentation and Lesion Characterization

Application direction. Prototype support tool for dermatology image triage and lesion measurement.
Required outcome. Design a system that segments skin lesions from dermoscopic or clinical images and computes interpretable lesion properties such as area, boundary irregularity, or color variation.

System Requirement Specification	• Produce a lesion mask and boundary overlay for each test image. • Compute at least two interpretable lesion descriptors. • Compare at least two segmentation methods, at least one of them classical. • Discuss clinical usefulness and limitations without making medical claims. • Provide a clear visual summary for representative success and failure cases.
Suggested Pipeline	1. Color normalization and artifact handling 2. Lesion segmentation 3. Boundary refinement 4. Morphological measurement 5. Optional lesion-type classification 6. Visual report generation
Required Data	ISIC challenge data or another lesion dataset. Use a clearly defined train/validation/test split; at least several hundred images are recommended.
Recommended Method Comparisons	• Segmentation: Otsu/adaptive threshold vs K-means in color space vs watershed vs lightweight U-Net baseline (optional) • Post-processing: morphology and contour smoothing vs CRF-like refinement if explored
Required Analysis and Evaluation in the Report	Use Dice, IoU, boundary quality, and descriptor stability. Discuss artifacts such as hair, ruler marks, low contrast, and irregular pigmentation. If classification is attempted, separate segmentation performance from diagnosis claims.

Project 6. Traffic Sign Detection and Recognition

Application direction. Road-scene assistance module for driver support or traffic-sign auditing.
Required outcome. Create a system that locates traffic signs in images and predicts their category or specific sign class.

System Requirement Specification	• Detect and label traffic signs on test images. • Compare at least two recognition strategies, one of which should be non-deep. • Evaluate both detection/localization and final classification quality. • Handle small signs or challenging lighting in at least a subset of experiments. • Document common confusion pairs and missed signs.
Suggested Pipeline	1. Color/contrast preprocessing 2. Candidate generation

	3. Region validation 4. Feature extraction 5. Classification 6. Output visualization with label and confidence
Required Data	GTSRB/GTSDB or another public traffic-sign dataset. Students may also add local images to test domain shift.
Recommended Method Comparisons	• Candidate generation: color filtering + shape cues vs sliding-window/region proposals • Recognition: HOG+SVM vs ORB+matching vs lightweight CNN • Optional domain-shift test on locally collected images
Required Analysis and Evaluation in the Report	Report precision/recall, class accuracy, and per-class confusion. Analyze failure cases due to blur, scale, occlusion, and sign similarity. Explain whether candidate-generation errors or classifier errors dominate.

Project 7. Lane Detection for Driver Assistance

Application direction. Prototype lane-marking perception module for ADAS-style road assistance.
Required outcome. Develop a system that identifies lane boundaries from road images or short video clips and overlays stable lane estimates.

System Requirement Specification	• Detect left and right lane boundaries in images or video frames. • Overlay lane estimates and optionally compute lane-center deviation. • Compare at least two extraction/detection pipelines. • Evaluate on one difficult condition such as worn paint, shadows, or curves. • Provide temporal stabilization if video is used.
Suggested Pipeline	1. Camera-frame preprocessing 2. Region-of-interest selection 3. Edge or line evidence extraction 4. Lane candidate generation 5. Curve fitting / temporal smoothing 6. Visualization and deviation summary
Required Data	TuSimple, CULane subset, or self-recorded dashcam data. Use both easy daytime scenes and at least one challenging scenario.
Recommended Method Comparisons	• Edge extraction: Sobel vs Canny • Lane detection: Hough transform vs polynomial fitting on bird's-eye view • Optional learning-based comparison with a lightweight lane model or segmentation baseline
Required Analysis and Evaluation in the Report	Discuss pixel-level or line-level accuracy, frame stability, and runtime. Examine failure under shadows, merges, curves, and missing markings. Separate geometric assumptions from actual detection errors.

Project 8. Panorama Stitching and Image Registration

Application direction. Mobile photography enhancement and virtual scene capture.
Required outcome. Build a system that stitches overlapping images into a panorama using feature matching and geometric alignment.

System Requirement Specification	• Successfully stitch multi-image sets into a visually coherent panorama. • Compare at least two feature-description strategies. • Use a robust outlier-removal method such as RANSAC or an equivalent.

	• Show both intermediate matches and final stitched output. • Document scenes where parallax or low texture causes failure.
Suggested Pipeline	1. Feature detection 2. Descriptor extraction 3. Feature matching 4. Outlier rejection 5. Homography estimation 6. Warping and blending 7. Panorama quality assessment
Required Data	Self-captured image sets from at least 30 scenes, with each scene containing overlapping images. Include some indoor and outdoor settings.
Recommended Method Comparisons	• Features/descriptors: SIFT-like (if available) vs ORB • Matching: brute force vs FLANN-like indexing • Blending: simple overlay vs feathering/multi-band style approximation
Required Analysis and Evaluation in the Report	Evaluate alignment quality, ghosting, seam visibility, and runtime. Explain the role of keypoint density, match quality, and planar-scene assumptions. Include representative success/failure case studies.

Project 9. License Plate Detection and Character Segmentation

Application direction. Entrance control, parking management, and toll-lane prototyping.
Required outcome. Create a system that finds license plates in vehicle images and segments characters or prepares crops for OCR.

System Requirement Specification	• Detect license-plate regions and visualize them. • Implement a plate-candidate filtering strategy with clear geometric heuristics or learned scoring. • Perform character segmentation or provide a prepared plate crop suitable for OCR. • Compare at least two localization approaches. • Discuss legal/privacy considerations in data handling.
Suggested Pipeline	1. Preprocessing and scale normalization 2. Plate candidate generation 3. Candidate filtering 4. Plate localization 5. Character segmentation or OCR preparation 6. Visualization and confidence logging
Required Data	CCPD or a simpler public license-plate dataset, optionally mixed with local images. If full recognition is difficult, character segmentation may be the main endpoint.
Recommended Method Comparisons	• Localization: edge+morphology vs contour-based filtering vs learning-based detector baseline • Character stage: vertical projection vs connected components vs OCR-ready crop only
Required Analysis and Evaluation in the Report	Measure localization precision/recall and, if character segmentation is included, segmentation accuracy or downstream OCR readiness. Analyze sensitivity to plate tilt, low resolution, night scenes, and reflections.

Project 10. Parking Occupancy Detection for Smart Parking

Application direction. Occupancy monitoring system for parking lots or campus parking dashboards.
Required outcome. Develop a system that predicts whether predefined parking slots are occupied from still images or video frames.

System Requirement Specification	• Determine occupancy for each parking slot in the scene. • Visualize slot status using colored overlays or a simple dashboard. • Compare at least two classification approaches. • Analyze at least one challenge such as rain, shadows, or nighttime scenes. • Support batch evaluation on a folder or video sequence.
Suggested Pipeline	1. Slot definition or slot detection 2. Patch extraction 3. Feature extraction or learned encoding 4. Occupancy classification 5. Visualization and summary dashboard
Required Data	PKLot or CNRPark+EXT, or a self-curated lot with manually defined slot regions. Use images from different weather/light conditions if available.
Recommended Method Comparisons	• Classifiers: intensity/texture features + SVM vs random forest vs lightweight CNN • Background handling: static ROI only vs normalization under weather or illumination
Required Analysis and Evaluation in the Report	Report per-slot accuracy, precision/recall, and confusion between empty, shadowed, and partially occluded slots. Discuss domain shift between lots or camera viewpoints.

Project 11. Waste Item Classification and Sorting Assistant

Application direction. Recycling support kiosk or educational waste-sorting assistant.
Required outcome. Build a system that classifies waste items from images into categories such as paper, plastic, metal, glass, or organic.

System Requirement Specification	• Predict waste category for each test image and show the result clearly. • Compare at least one classical method with one learning-based method. • If multiple objects appear, either isolate one item or justify single-object assumptions. • Provide confusion analysis for visually similar categories. • Present the system as an application with a clear end user.
Suggested Pipeline	1. Image preprocessing 2. Object isolation or crop extraction 3. Feature extraction or representation learning 4. Category classification 5. User-facing result display and confidence explanation
Required Data	TrashNet or similar public waste datasets, optionally extended with local photos to test scene variation.
Recommended Method Comparisons	• Classical: color/texture/shape features + SVM/random forest • Learning-based: transfer learning with a lightweight CNN • Optional hybrid: segmentation before classification vs direct classification
Required Analysis and Evaluation in the Report	Use accuracy, macro-F1, and a confusion matrix. Discuss ambiguous items, contamination, damaged packaging, and viewpoint sensitivity. Explain which classes benefit most from segmentation or texture cues.

Project 12. Fruit Quality Grading and Ripeness Assessment

Application direction. Prototype fruit-sorting aid for retail, packaging, or farm quality control.
Required outcome. Create a system that detects fruit, measures relevant appearance cues, and predicts ripeness or quality grade.

System Requirement Specification	• Segment the fruit or justify crop-based processing. • Predict at least two quality levels or ripeness stages. • Compare at least two grading strategies. • Show interpretable evidence such as color distribution or detected blemish regions. • Frame the solution as a usable quality-control tool.
Suggested Pipeline	1. Fruit segmentation 2. Color correction 3. Feature extraction (color, size, texture, blemish cues) 4. Grade/ripeness classification 5. Result visualization and summary statistics
Required Data	Self-collected fruit images or a public fruit dataset. Include multiple ripeness stages and visible defects if possible.
Recommended Method Comparisons	• Segmentation: threshold in HSV/Lab vs contour-based separation • Classification: rule-based grading vs SVM/random forest vs lightweight CNN • Optional quality metrics: bruise-region extraction
Required Analysis and Evaluation in the Report	Report classification accuracy, per-grade confusion, and sensitivity to lighting and fruit variety. Discuss whether handcrafted features remain stable across imaging conditions.

Project 13. Fabric Defect Detection in Textile Inspection

Application direction. Automated visual inspection support for textile manufacturing lines.
Required outcome. Develop a system that detects anomalous or defective fabric regions and highlights them for inspection.

System Requirement Specification	• Detect defect regions and mark them on the image. • Compare at least two texture-analysis strategies. • Measure performance on subtle defects as well as obvious ones. • Provide a notion of defect severity or area if possible. • Explain why the chosen features are appropriate for repetitive texture.
Suggested Pipeline	1. Texture normalization 2. Patch extraction or full-image analysis 3. Texture feature computation 4. Defect-region detection 5. Visualization and severity summary
Required Data	AIDA/fabric defect datasets or a curated set of patterned fabric images with defect labels.
Recommended Method Comparisons	• Texture-based: GLCM/LBP/Gabor features + anomaly classifier • Segmentation-based: thresholding in filtered domain vs clustering • Learning-based baseline optional for comparison
Required Analysis and Evaluation in the Report	Use precision/recall, IoU for defect regions if masks are available, and defect/no-defect accuracy. Analyze sensitivity to periodic texture, illumination gradients, and tiny defects.

Project 14. Surface Defect Detection on Industrial Materials

Application direction. Visual quality inspection for metal, ceramic, or manufactured surfaces.
Required outcome. Build a system that identifies scratches, dents, stains, or irregular patterns on surface images.

System Requirement Specification	• Detect or classify at least three defect types or defect/no-defect states. • Compare at least two feature/modeling approaches. • Visualize defect evidence using boxes, masks, or heat-style overlays. • Discuss deployment constraints such as speed and false alarms. • Include a clear manufacturing-oriented use case.
Suggested Pipeline	1. Illumination normalization 2. Feature extraction or filtered image generation 3. Defect localization or patch classification 4. Post-processing 5. Defect report generation
Required Data	NEU surface defect dataset or another industrial surface dataset with defect labels.
Recommended Method Comparisons	• Classical texture features vs filtered residuals vs lightweight CNN baseline • Localization: sliding-window scoring vs connected-component extraction from anomaly maps
Required Analysis and Evaluation in the Report	Report class-wise accuracy or localization metrics, plus false-alarm behavior. Examine confusion between scratches, stains, and normal texture changes. Discuss practical tolerance settings.

Project 15. Hand Gesture Recognition for Human–Computer Interaction

Application direction. Touchless control interface for slides, kiosks, or smart devices.
Required outcome. Create a system that segments the hand and predicts a gesture category from images or short clips.

System Requirement Specification	• Recognize a fixed set of gestures and map them to interface commands. • Compare at least one contour/shape-based pipeline with one learned approach. • Address background clutter or illumination in the hand-segmentation stage. • Demonstrate the system in a small real interaction scenario. • Provide confusion analysis for similar gestures.
Suggested Pipeline	1. Hand-region extraction 2. Contour or key-shape analysis 3. Feature extraction or temporal encoding 4. Gesture classification 5. Output display and control command mapping
Required Data	LeapGestRecog, a webcam-collected dataset, or another public gesture dataset. Ensure gesture classes are balanced.
Recommended Method Comparisons	• Classical: skin-color segmentation + Hu moments/contour features + classifier • Learning-based: lightweight CNN on cropped hands • Optional temporal comparison if video is used
Required Analysis and Evaluation in the Report	Report accuracy, latency, and robustness to pose, scale, and background. Discuss false positives when non-hand regions have similar color or shape.

Project 16. Building Footprint Extraction from Aerial Images

Application direction. Urban mapping support and basic land-use analysis.
Required outcome. Design a system that extracts building regions from aerial or satellite images and estimates simple statistics such as area coverage.

System Requirement Specification	• Segment building footprints and visualize them over the original image. • Compute at least one useful statistic such as building count or area ratio. • Compare at least two segmentation strategies or post-processing variants. • Discuss scale, roof color, and shadow issues. • Frame the project as a geospatial application, not only a benchmark exercise.
Suggested Pipeline	1. Preprocessing and tiling 2. Building-region segmentation 3. Morphological cleanup 4. Connected-region analysis 5. Map overlay and area statistics
Required Data	Inria Aerial Image Labeling dataset or a smaller public aerial-building dataset. Students should use a manageable subset if compute is limited.
Recommended Method Comparisons	• Classical: thresholding/clustering on multispectral or RGB cues + morphology • Learning-based baseline optional if resources allow • Post-processing: shape filtering and region merging
Required Analysis and Evaluation in the Report	Use IoU/Dice if masks are available, plus region-count differences. Analyze false merges, missed small roofs, and confusion with roads or parking lots.

Project 17. Crowd Density Estimation and Crowd-Level Monitoring

Application direction. Public-space safety monitoring and event planning support.
Required outcome. Construct a system that estimates crowd density or person counts from images and highlights dense-risk areas.

System Requirement Specification	• Produce crowd estimate or density-level output for each image. • Visualize dense areas or person-evidence regions. • Compare at least two estimation strategies, with one simplified baseline. • Discuss ethical limits and privacy in surveillance-style applications. • Focus on methodological clarity rather than claiming deployable safety decisions.
Suggested Pipeline	1. Scene preprocessing 2. Person evidence extraction or density estimation 3. Crowd-level summarization 4. Risk-zone visualization 5. Statistical output
Required Data	ShanghaiTech or a smaller crowd dataset. If exact counting is too difficult, density-level classification may be the primary target.
Recommended Method Comparisons	• Classical baseline: foreground/feature density proxies • Learning-based: lightweight density-estimation model or transfer baseline • Alternative target: low/medium/high crowd-level classification
Required Analysis and Evaluation in the Report	Use MAE/MSE for count estimation or accuracy/F1 for density levels. Discuss occlusion, scale variation, and perspective distortion. Be explicit about what the model can and cannot infer.

Project 18. Food Recognition and Portion-Aware Dietary Assistant

Application direction. Diet logging and meal-analysis prototype for personal nutrition support.
Required outcome. Build a system that classifies food items from meal images and optionally estimates coarse portion categories or calorie ranges.

System Requirement Specification	• Predict food category for each image or crop. • If portion estimation is included, define a coarse and honest target such as small/medium/large. • Compare at least one baseline against one stronger method. • Present outputs in an application-oriented way suitable for a user diary or meal summary. • Discuss ambiguity caused by mixed dishes and plating variation.
Suggested Pipeline	1. Food-region extraction or crop assumption 2. Feature extraction or transfer learning 3. Food-category classification 4. Optional portion-size approximation 5. Meal summary visualization
Required Data	Food-101 or a smaller curated subset; students may add locally captured meal photos for domain-shift testing.
Recommended Method Comparisons	• Classical features + SVM as baseline • Transfer learning with a lightweight CNN • Optional segmentation-first vs direct classification comparison
Required Analysis and Evaluation in the Report	Use top-1 accuracy, macro-F1, and qualitative analysis on mixed foods. If portion size is estimated, clearly state the simplified assumptions. Discuss realistic application limits.

Project 19. Medical Image Abnormality Localization for Educational Triage

Application direction. Educational prototype for highlighting suspicious regions in medical images without replacing diagnosis.
Required outcome. Develop a system that localizes candidate abnormal regions in a medical image domain such as chest X-ray, bone X-ray, or dermoscopic imagery, and presents interpretable evidence.

System Requirement Specification	• Choose a clearly limited medical task and justify it. • Localize suspicious regions or produce an interpretable candidate score. • Compare at least two approaches, with explicit limitations. • Avoid unsupported medical claims and include an ethics/responsibility note. • Present results in a way suitable for education or algorithm study.
Suggested Pipeline	1. Domain-specific preprocessing 2. Region-of-interest detection or segmentation 3. Candidate abnormality scoring/classification 4. Visual explanation generation 5. Case-level summary
Required Data	Choose one public dataset appropriate for the domain and scope. The project should remain modest and should not promise clinical deployment.
Recommended Method Comparisons	• Classical ROI and feature pipelines vs lightweight learning-based baseline • Explanation: heatmap/saliency-style visualization or region highlighting • Students should keep scope narrow and realistic
Required Analysis and Evaluation in the Report	Use task-appropriate metrics such as classification accuracy, IoU, or sensitivity/specificity where justified. Emphasize error analysis and uncertainty. Discuss annotation noise and domain shift.

Project 20. Road Damage Detection for Smart Infrastructure Inspection

Application direction. Municipal road-maintenance support and visual auditing of road conditions. Required outcome. Create a system that finds road damage such as cracks or potholes in road-surface images and categorizes severity or type.

System Requirement Specification	• Detect or segment road damage regions and visualize them. • Classify at least crack vs pothole vs normal, or justify a simpler label set. • Compare at least two approaches, one of them classical. • Discuss practical deployment issues such as motion blur, shadows, and viewpoint changes. • Frame the output as an infrastructure-support tool.
Suggested Pipeline	1. Road-surface preprocessing 2. Damage-region enhancement 3. Damage detection or segmentation 4. Type/severity classification 5. Map-ready output or maintenance summary
Required Data	RDD2022 or a similar road-damage dataset. Students may also collect local smartphone imagery for qualitative testing.
Recommended Method Comparisons	• Classical: edge/texture-based crack enhancement + morphology • Learning-based: lightweight detector/segmenter baseline • Optional severity scoring by region size or confidence
Required Analysis and Evaluation in the Report	Use precision/recall, IoU where possible, and false-alarm analysis on non-damage textures. Discuss what kinds of road appearance patterns are frequently mistaken for damage.

Appendix. Course Project Report Template

Objective of the appendix. This section provides a unified report structure for students to submit their projects in a technical/academic style. The report may be written in Vietnamese or English, but terminology, figure captions, table captions, and the presentation of results must be consistent throughout.

A. Formatting Information

- The report should be written in the form of a technical essay, with margins of top – bottom – left – right = 2 – 2 – 2.5 – 2 cm.
- Recommended length: 15–20 A4 pages excluding the source-code appendix, no fewer than 10 pages and no more than 35 pages; base font size 13 – Times New Roman – line spacing from 1.3 to 1.5 – fully justified alignment – the report may be prepared in LaTeX or MS Word.
- Figures and tables must have titles, numbering, and must be referred to in the main text.
- Source code must not be inserted directly into the report; only algorithms or pseudo-code may be presented. Numerical results should be presented in tables rather than screenshots. Mathematical formulas must be typeset rather than pasted as images.
- Submit the source code or a repository link, a file explaining how to run the project, and at least 5 images illustrating the system outputs.

B. Suggested Report Structure

1. Title, names and student IDs of group members, class, and instructor.
2. Abstract: state the problem, application, dataset, main methods, and key results.
3. Main Content:

Introduction: motivation for the problem, application context, project objectives, and scope.

Chapter (Section) 1. Theoretical Background (or Methods and Data)

1.1. Methods: describe the complete pipeline, explain each processing block, and present at least 2 methods for comparison in one main task.

1.2. Data and preprocessing: describe the data source, sampling strategy, labeling conventions, dataset split, and cleaning steps.

Chapter (Section) 2. Experiments and Evaluation

2.1. Experimental setup: runtime configuration, experimental scenarios, and evaluation metrics

2.2. Experiments: present the experiments for each method in sequence, together with result tables and illustrative output figures.

2.3. Analysis and discussion: error analysis, good/bad cases, reasons why one method performs better or worse than another, and system limitations.

Conclusion and future work: clearly state what has been achieved and the feasible directions for improvement.

References: use a standardized citation style.

Appendix (if needed): interface description, additional qualitative results, and description of the source-code folder structure.

C. Mandatory Content Checklist

- An overall pipeline diagram is included.
- The dataset and the dataset split are clearly described.
- At least one method-comparison table is included.
- At least one error-analysis section is included.
- A conclusion on the practical applicability of the system is included.